

Guia de Estudo — Apresentação do Trabalho de BD (MOD 02)

Esse arquivo serve para você dominar **o que, onde, e como** cada peça do projeto foi implementada. Ele responde às perguntas mais prováveis da professora sobre código, conexão Java↔MySQL, frontend e cada item das Etapas 04, 05 e 06.

0. Resumo em uma linha

“É um sistema desktop em Java Swing que conversa com um banco MySQL usando JDBC puro. Toda a SQL é escrita à mão dentro das classes DAO, sem ORM. A interface tem abas de CRUD para 5 tabelas, mais abas para consultas, views, funções, procedimentos, triggers e um dashboard com gráficos.”

1. Como o Java se conecta ao MySQL (a pergunta favorita da professora)

1.1 O ponto único de conexão

Arquivo: `src/main/java/conexao/ConexaoBanco.java`

```
public class ConexaoBanco {  
    private static final String URL_BANCO =  
    "jdbc:mysql://localhost:3306/supermercado";  
    private static final String USUARIO    = "root";  
    private static final String SENHA      = "root";  
  
    public static Connection obterConexao() throws SQLException {  
        return DriverManager.getConnection(URL_BANCO, USUARIO, SENHA);  
    }  
}
```

O que está acontecendo aqui, linha por linha:

- `import java.sql.Connection, DriverManager, SQLException` → tudo isso vem do **JDBC padrão da linguagem Java** (`java.sql.*`).
- `URL_BANCO` segue o formato do JDBC: `jdbc:<sgbd>://<host>:<porta>/<nome_do_banco>`. No nosso caso: `jdbc:mysql://localhost:3306/supermercado`.
- `DriverManager.getConnection(url, user, senha)` é o método do JDBC que **encontra o driver no classpath, carrega-o e abre uma sessão TCP com o MySQL**.
- Quem é o "driver" carregado? É a biblioteca **mysql-connector-j** declarada no `pom.xml`. Sem ela, `getConnection` não saberia conversar com MySQL.
- Não há nenhuma camada acima do JDBC** — nenhum ORM, nenhum mapeamento, nenhuma geração de SQL. O Java envia a SQL crua para o banco.

1.2 Como cada DAO usa essa conexão

Padrão usado em **todos** os DAOs (**ClienteDAO**, **ProdutoDAO**, ...):

```
String sql = "INSERT INTO cliente (cpf, nome, telefone) VALUES (?, ?, ?)";

try (Connection conexao = ConexaoBanco.obterConexao();
    PreparedStatement ps = conexao.prepareStatement(sql)) {

    ps.setString(1, cliente.getCpf());
    ps.setString(2, cliente.getNome());
    ps.setString(3, cliente.getTelefone());
    ps.executeUpdate();
}
```

Passo a passo:

1. **ConexaoBanco.obterConexao()** abre uma conexão (passo 1.1).
2. **prepareStatement(sql)** envia a string SQL para o MySQL com placeholders `?`. O banco compila a query e devolve um plano de execução. Isso protege contra **SQL Injection**.
3. **ps.setString(i, valor)** preenche cada `?` na ordem. Existe **setInt**, **setBigDecimal**, **setDate**, etc.
4. **executeUpdate()** envia o INSERT/UPDATE/DELETE; **executeQuery()** envia um SELECT e devolve um **ResultSet**.
5. **try-with-resources** fecha automaticamente o **PreparedStatement** e a **Connection** ao sair do bloco — sem precisar de **finally**.

1.3 Por que isso atende ao requisito da professora

O enunciado proíbe ORM e exige “SQL explícita no backend”. No nosso código:

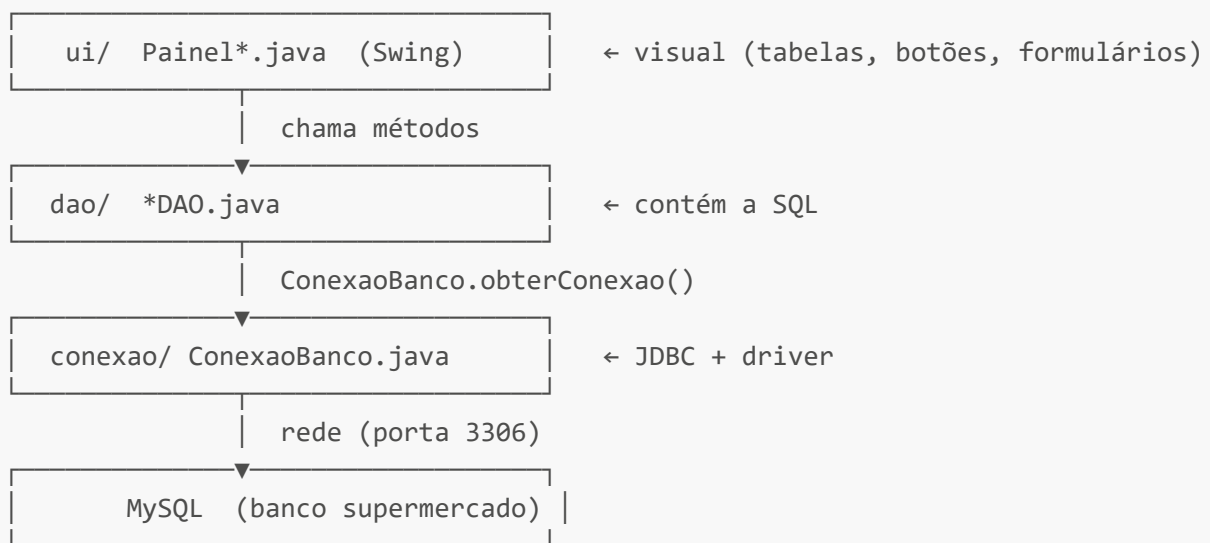
- O SQL aparece **literalmente como String** dentro de cada método DAO.
- Não há geração automática de SQL.
- A única biblioteca relacionada a banco é o driver MySQL — **isto é o que faz a JVM falar com o servidor MySQL**, não é um ORM.

1.4 Mapa “onde acontece o SQL”

Classe Java	Tabela / Operação principal
ClienteDAO	cliente — INSERT, SELECT, UPDATE, DELETE
ProdutoDAO	produto — CRUD + alterar preço (dispara trigger)
FilialDAO	filial — CRUD
FuncionarioDAO	funcionario — CRUD (com auto-FK supervisor)
VendaDAO	vende + vende_produto — CRUD + itens
RelatoriosDAO	views, funções, procedimentos, logs, dashboard

2. Como o Frontend (Swing) se liga ao Backend

2.1 Arquitetura em 3 camadas



2.2 Exemplo concreto: o botão "Inserir" da aba Clientes

Em `ui/PainelClientes.java`:

```
btInserir.addActionListener(e -> abrirFormulario(null));
```

`abrirFormulario(null)` mostra um `JOptionPane` com 3 campos. Quando o usuário clica OK:

```

Cliente c = new Cliente(
    campoCpf.getText().trim(),
    campoNome.getText().trim(),
    campoTelefone.getText().trim());
dao.inserir(c);    // ← aqui o painel chama a camada DAO
recarregar();      // recarrega a JTable com SELECT
  
```

`dao.inserir(c)` está em `ClienteDAO.java` e executa o `INSERT` mostrado em 1.2.

2.3 Como a tabela na tela é atualizada

`recarregar()` faz:

```

List<Cliente> lista = dao.listar();    // SELECT cpf, nome, telefone FROM cliente
modelo.setRowCount(0);
for (Cliente c : lista) modelo.addRow(...);
  
```

`DefaultTableModel` é a estrutura interna do `JTable`. Setamos as linhas e o Swing renderiza automaticamente.

3. Etapa 03 — Modelo, criação e inserção

3.1 Constraints e decisões

Tabela / coluna	Constraint	Por quê
<code>filial.cnpj</code>	<code>CHECK (LENGTH=14)</code>	CNPJ tem 14 dígitos
<code>filial_telefone.cnpj_filial</code>	<code>ON UPDATE CASCADE</code>	Se mudarmos o CNPJ da filial, telefones acompanham
<code>funcionario.cpf</code>	<code>UNIQUE</code>	Um CPF aparece em no máximo um cadastro
<code>funcionario.tipo</code>	<code>CHECK IN (operacional, administrativo)</code>	Domínio fechado
<code>funcionario.supervisor</code>	<code>ON DELETE SET NULL</code>	Ao remover supervisor, subordinado fica sem chefe (não some)
<code>produto.preco_base</code>	<code>CHECK ≥ 0</code>	Não existe preço negativo
<code>vende.cpf_cliente</code>	<code>ON DELETE SET NULL</code>	Ao apagar cliente, histórico da venda é preservado
<code>vende.pagamento</code>	<code>CHECK IN (...)</code>	Restringe formas de pagamento aceitas
<code>vende.data_venda</code>	<code>DEFAULT (CURRENT_DATE)</code>	Data atual quando não informada
<code>entrega.numero_entrega</code>	<code>AUTO_INCREMENT</code>	Sequência automática
<code>vende_produto.quantidade</code>	<code>CHECK > 0</code>	Não vende quantidade zero ou negativa
<code>departamento.categoria</code>	<code>UNIQUE</code>	Não pode haver duas categorias iguais

3.2 Tabela "fraca"

`dependente` é fraca porque sua existência depende do funcionário. Sua chave primária é **composta** (`matricula`, `mat_func`).

3.3 Auto-relacionamento

`funcionario.supervisor` referencia a própria `funcionario.matricula`. Isso permite hierarquia de chefia.

3.4 N:N

- `vende_produto` (relaciona `vende` × `produto`)

- departamento_produto (relaciona departamento × produto)

4. Etapa 04 — Consultas, Views e Índices

Arquivo: `sql/03_consultas_views_indices.sql`. Os mesmos comandos são executados pelo Java em `RelatoriosDAO` e exibidos na aba "Consultas/Views".

4.1 Consultas

#	Tipo	Pergunta de negócio
1	JOIN + GROUP BY + HAVING	Funcionários com mais de uma venda
2	2 JOINS + WHERE	Vendas em outubro/2025 com nome do cliente e funcionário
3	LEFT JOIN ... IS NULL (anti join)	Cientes que nunca compraram
4	SUBCONSULTA	Produtos com preço acima da média geral

Como explicar ANTI JOIN: "É um LEFT JOIN seguido de `WHERE chave_da_direita IS NULL`. Pega os registros da esquerda que não têm correspondência na direita."

Como explicar HAVING vs WHERE: "WHERE filtra antes do agrupamento, HAVING filtra **depois** do GROUP BY (sobre o resultado das funções de agregação)."

4.2 Views

View	Fórmula	Justificativa
<code>vw_vendas_detalhadas</code>	3 JOINS + WHERE	Reúne NF-e, cliente, funcionário e produto numa só consulta reutilizável.
<code>vw_funcionarios_destaque</code>	1 JOIN + subconsulta	Funcionários cujo total de vendas é ≥ média geral. Encapsula o cálculo.

Como explicar view: "É uma consulta nomeada que se comporta como uma tabela. Não armazena dados, mas centraliza lógica de SELECT que se repetiria em vários lugares."

4.3 Índices

Índice	Coluna	Por que
<code>idx_vende_matricula_func</code>	<code>vende.matricula_func</code>	Acelera o GROUP BY e os JOINS por funcionário (consulta 1.1, view destaque).
<code>idx_vende_cpf_cliente</code>	<code>vende.cpf_cliente</code>	Acelera o anti-join (consulta 1.3) e a view detalhada.

Como explicar índice: "É uma estrutura B-tree paralela à tabela. Sem índice, o MySQL faz *full table scan* (lê tudo). Com índice, ele busca a chave em $O(\log n)$ e pula direto para as linhas relevantes. Em troca, INSERT/UPDATE ficam um pouco mais caros porque o índice precisa ser mantido."

5. Etapa 05 — Funções, Procedimentos e Triggers

Arquivo: `sql/04_funcoes_procedimentos_triggers.sql`.

5.1 Funções

Função	O que retorna	Estrutura especial
<code>fn_total_venda(nfe)</code>	DECIMAL — total de uma venda	só <code>SELECT SUM(...)</code>
<code>fn_categoria_funcionario(mat)</code>	VARCHAR — "Ouro" / "Prata" / "Bronze"	IF / ELSEIF / ELSE

Como explicar função: "Função sempre devolve um valor (RETURN). Pode ser usada dentro de um SELECT como se fosse uma coluna calculada."

A regra do `fn_categoria_funcionario`:

- vendas ≥ 5 → Ouro
- vendas ≥ 2 → Prata
- menos → Bronze

5.2 Procedimentos

Procedimento	O que faz	Tipo
<code>pr_atualizar_preco_departamento(d, p)</code>	UPDATE em todos os produtos do departamento, multiplicando preço por $(1 + p\%)$	UPDATE
<code>pr gerar_resumo_funcionarios()</code>	Para CADA funcionário, calcula total de vendas, soma de valor e classificação. Insere uma linha em <code>resumo_vendas_funcionario</code> .	CURSOR

Por que precisa de cursor em `pr gerar_resumo_funcionarios`? Porque a saída é UMA linha por funcionário em uma TABELA NOVA, com cálculo próprio por linha (chamando uma função para classificar). Um único UPDATE não consegue gerar uma tabela de resumo nova com lógica linha a linha.

Como explicar cursor: "É um ponteiro de iteração sobre o resultado de uma query. Você abre o cursor, faz FETCH para pegar a próxima linha em variáveis, processa, e fecha quando termina. É equivalente a um `for each` em PL/SQL."

5.3 Triggers

Trigger	Quando dispara	O que faz
<code>tg_log_nova_venda</code>	AFTER INSERT ON vende	Insere linha em <code>log_alteracoes</code> registrando a venda
<code>tg_log_alteracao_preco</code>	AFTER UPDATE ON produto	Se preço mudou, registra preço antigo + novo em <code>log_alteracoes</code>

Como explicar trigger: "É um gatilho. O banco executa o bloco automaticamente quando uma operação (INSERT/UPDATE/DELETE) acontece em uma tabela. Não é chamado pelo Java — o Java só faz INSERT/UPDATE comum, e o trigger 'pega carona' para criar o log."

Como mostrar o trigger funcionando ao vivo:

1. Vá na aba **Vendas** → clique **Inserir**.
2. Crie uma NF-e nova qualquer.
3. Vá na aba **Logs** → clique **Atualizar logs**.
4. A nova entrada aparece (gerada pelo trigger).

6. Etapa 06 — Interface funcional

6.1 CRUD para 5 tabelas (requisito ≥ 4)

- **Clientes** (`PainelClientes` ↔ `ClienteDAO`)
- **Produtos** (`PainelProdutos` ↔ `ProdutoDAO`)
- **Funcionários** (`PainelFuncionarios` ↔ `FuncionarioDAO`)
- **Filiais** (`PainelFiliais` ↔ `FilialDAO`)
- **Vendas** (`PainelVendas` ↔ `VendaDAO`, com itens)

Cada painel tem botões **Inserir**, **Editar**, **Excluir**, **Atualizar lista** e abre o formulário em `JOptionPane.showConfirmDialog`.

6.2 Integração com Funções, Procedimentos e Triggers

- Aba **Funções/Procedimentos** chama:
 - `fn_total_venda(?)` via `SELECT fn_total_venda(?)` (`PreparedStatement`).
 - `fn_categoria_funcionario(?)` da mesma forma.
 - `pr_atualizar_preco_departamento(?, ?)` via `CallableStatement` (`{ CALL pr_xxx(?, ?)`).
 - `pr gerar resumo funcionarios()` via `CallableStatement`.
- Aba **Logs** mostra a tabela `log_alteracoes`, alimentada pelos triggers.

Sobre `CallableStatement`: "É a forma JDBC pura de chamar PROCEDURE. A sintaxe `{ CALL nome(?, ?) }` é o padrão JDBC para invocação de procedimentos no banco."

6.3 Consultas e Views na interface

Aba **Consultas/Views** tem 6 botões:

- 4 botões executam as consultas 1.1–1.4 (Etapa 04).
- 2 botões mostram as views.
- A consulta de "vendas no período" tem **dois campos de filtro** (data inicial e data final) — exemplo do requisito "filtros e parâmetros quando aplicável".

Cada resultado é renderizado em um `JTable` dinâmico (montado pelo `TabelaUtil.construirModelo`).

6.4 Dashboard (extra +0,5)

Aba **Dashboard** mostra:

- **5 indicadores numéricos:** total de clientes, produtos, funcionários, vendas; preço médio dos produtos.
- **5 gráficos JFreeChart** (todos vindos de SQL):
 1. Vendas por funcionário (barras)
 2. Vendas por forma de pagamento (pizza)
 3. Vendas por dia (linha)
 4. Top 5 produtos mais vendidos (barras)
 5. Vendas por filial (barras)

Botão **Atualizar dashboard** roda novamente todas as queries.

JFreeChart é "ferramenta para facilitar BD"? Não. JFreeChart é apenas para desenhar gráficos. Os dados vêm exclusivamente de SQL feita por nós em `RelatoriosDAO.dashboardXxx()`.

7. Perguntas-armadilha que a professora pode fazer

7.1 "Onde exatamente o SQL é enviado para o banco?"

Em cada DAO, dentro do método. O método `prepareStatement(sql)` empacota a String para o servidor; `executeUpdate()` ou `executeQuery()` faz a chamada de fato pela rede. O endereço (host, porta, banco) está em `ConexaoBanco.URL_BANCO`.

7.2 "Por que `PreparedStatement` em vez de `Statement`?"

1. **Segurança:** parâmetros ficam fora da string SQL → previne SQL Injection.
2. **Performance:** o banco compila o plano de execução uma vez.
3. **Tipos:** `setInt`, `setDate`, `setBigDecimal` cuidam da conversão, sem precisar concatenar string.

7.3 "Como o trigger é disparado pela aplicação?"

A aplicação **não** dispara o trigger explicitamente. Ela faz um INSERT normal (em `vende`, por exemplo). O **MySQL** detecta que existe um `AFTER INSERT` cadastrado e executa o bloco automaticamente, dentro da mesma transação.

7.4 "Por que esse procedimento precisa de cursor?"

Porque ele gera **uma linha por funcionário** numa **tabela diferente** (`resumo_vendas_funcionario`), com a classificação da função `fn_categoria_funcionario` aplicada por linha. Um único UPDATE/INSERT SELECT não conseguiria, porque a classificação depende de uma chamada de função PL/SQL para CADA matrícula.

7.5 "E se o MySQL estiver fora do ar?"

O `Principal.main` faz um `try { ConexaoBanco.obterConexao(); }` antes de abrir a janela. Se falhar, mostra um `JOptionPane` com instrução para o usuário rodar os scripts e conferir credenciais.

7.6 "Onde está a 'identificação' de cada item para a professora?"

- **Conexão:** `conexao/ConexaoBanco.java`

- **CRUD:** `dao/*.java` (mesmo padrão em todos)
- **4 consultas:** `RelatoriosDAO.consulta*()` + `sql/03_*.sql` itens 1.x
- **2 views:** `RelatoriosDAO.view*()` + `sql/03_*.sql` itens 2.x
- **2 índices:** fim do `sql/03_*.sql`
- **2 funções:** `RelatoriosDAO.chamarFuncao*()` + `sql/04_*.sql`
- **2 procedimentos:** `RelatoriosDAO.chamarProcedimento*()` + `sql/04_*.sql`
- **2 triggers:** visualizáveis na aba Logs + `sql/04_*.sql`
- **Dashboard:** `ui/PainelDashboard.java` + `RelatoriosDAO.dashboard*()`

8. Roteiro de demonstração (6 minutos da apresentação 01)

1. **Modelo conceitual** (30s) — abrir o `brModelo`, mostrar entidades, fraca, especialização e cardinalidades.
2. **Modelo lógico** (30s) — mesma coisa, mostrar as FKs.
3. **App rodando** (30s) — abrir a janela principal, mostrar todas as abas.
4. **CRUD** (60s) — criar um cliente, alterar nome, deletar.
5. **Etapa 04** (60s) — abrir Consultas/Views, rodar as 4 consultas e as 2 views.
6. **Etapa 05** (90s) — chamar `fn_total_venda`, alterar preço de produto pela aba Produto, ir na aba Logs e mostrar a entrada do trigger; chamar o procedimento com CURSOR e mostrar a tabela de resumo populada.
7. **Dashboard** (60s) — virar para a aba Dashboard, mostrar os 5 gráficos e ressaltar que tudo veio do banco.

9. Checklist final antes de apresentar

- ☐ MySQL está rodando.
- ☐ Os 4 scripts SQL foram executados na ordem.
- ☐ `ConexaoBanco.java` tem as credenciais corretas.
- ☐ Aplicação abre sem erro de conexão.
- ☐ Todas as 9 abas carregam.
- ☐ CRUD de Cliente funciona.
- ☐ Aba Consultas mostra as 4 consultas e 2 views.
- ☐ Aba Funções/Procedimentos roda as 4 chamadas.
- ☐ Aba Logs mostra entradas dos dois triggers (depois de você ter inserido pelo menos 1 venda e alterado pelo menos 1 preço).
- ☐ Aba Dashboard mostra 5 gráficos e 5 indicadores.
- ☐ O slide do `brModelo` (conceitual + lógico) está aberto em outra aba do navegador.